



Problem Challenge 1

We'll cover the following



- Minimum Meeting Rooms (hard)
- Try it yourself

Minimum Meeting Rooms (hard)

Given a list of intervals representing the start and end time of 'N' meetings, find the **minimum number of rooms** required to **hold all the meetings**.

Example 1:

Meetings: `[[1,4], [2,5], [7,9]]`

Output: 2

Explanation: Since `[1,4]` and `[2,5]` overlap, we need two rooms to hold these two meetings. `[7,9]` can occur in any of the two rooms later.

Example 2:

Meetings: `[[6,7], [2,4], [8,12]]`

Output: 1

Explanation: None of the meetings overlap, therefore we only need one room to hold all meetings.

Example 3:

Meetings: `[[1,4], [2,3], [3,6]]`

Output: 2

Explanation: Since `[1,4]` overlaps with the other two meetings `[2,3]` and `[3,6]`, we need two rooms to hold all the meetings.

Example 4:

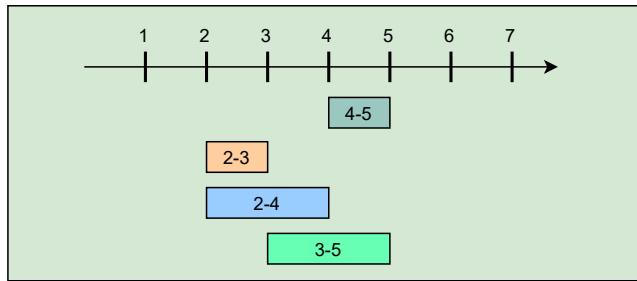
Meetings: `[[4,5], [2,3], [2,4], [3,5]]`

Output: 2

Explanation: We will need one room for `[2,3]` and `[3,5]`, and another room for `[2,4]` and `[4,5]`.

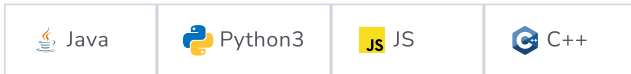
Here is a visual representation of Example 4:





Try it yourself

Try solving this question here:



```

1  import java.util.*;
2
3  class Meeting {
4      int start;
5      int end;
6
7      public Meeting(int start, int end) {
8          this.start = start;
9          this.end = end;
10     }
11 };
12
13 class MinimumMeetingRooms {
14
15     public static int findMinimumMeetingRooms(List<Meeting> meetings) {
16         // TODO: Write your code here
17         return -1;
18     }
19
20     public static void main(String[] args) {
21         List<Meeting> input = new ArrayList<Meeting>() {
22             {
23                 add(new Meeting(4, 5));
24                 add(new Meeting(2, 3));
25                 add(new Meeting(2, 4));
26                 add(new Meeting(3, 5));
27             }
28         };
29         int result = MinimumMeetingRooms.findMinimumMeetingRooms(input);
30         System.out.println("Minimum meeting rooms required: " + result);
31
32         input = new ArrayList<Meeting>() {
33             {
34                 add(new Meeting(1, 4));
35                 add(new Meeting(2, 5));
36                 add(new Meeting(7, 9));
37             }
38         };
39         result = MinimumMeetingRooms.findMinimumMeetingRooms(input);
40         System.out.println("Minimum meeting rooms required: " + result);
41
42         input = new ArrayList<Meeting>() {
43             {
44                 add(new Meeting(6, 7));
45                 add(new Meeting(2, 4));
46                 add(new Meeting(8, 12));
47             }
48         };
49         result = MinimumMeetingRooms.findMinimumMeetingRooms(input);

```

```
50 System.out.println("Minimum meeting rooms required: " + result);
51
52 input = new ArrayList<Meeting>() {
53     {
54         add(new Meeting(1, 4));
55         add(new Meeting(2, 3));
56         add(new Meeting(3, 6));
57     }
58 };
59 result = MinimumMeetingRooms.findMinimumMeetingRooms(input);
60 System.out.println("Minimum meeting rooms required: " + result);
61
62 input = new ArrayList<Meeting>() {
63     {
64         add(new Meeting(4, 5));
65         add(new Meeting(2, 3));
66         add(new Meeting(2, 4));
67         add(new Meeting(3, 5));
68     }
69 };
70 result = MinimumMeetingRooms.findMinimumMeetingRooms(input);
71 System.out.println("Minimum meeting rooms required: " + result);
72 }
73 }
74
```

[Run](#)[Save](#)[Reset](#)[← Back](#)[✓ Completed](#)[Next →](#)

Conflicting Appointments (medium)

Solution Review: Problem Challenge 1